PRD: Opening Dojo — London System & Caro-Kann Chess Trainer

1. Product Summary

Opening Dojo is a chess training web application focused on helping returning adult players learn a compact, practical opening repertoire:

- White offense: London System
- Black defense vs 1.e4: Caro-Kann Defense

The product teaches opening principles, drills move sequences, explains common plans, and lets users practice against realistic opponent responses.

The application should be suitable as an AI-assisted code generation demo using a Next.js / TypeScript stack. The demo should show strong software engineering discipline: domain modeling, validation, AI-assisted explanation, deterministic chess logic, and clean separation between rules, content, and AI.

2. Target User

Primary user:

A returning chess player who understands basic rules but does not want to memorize grandmaster-level theory.

User goals:

- Learn a simple opening repertoire
- Avoid early blunders
- Understand why moves are played
- Practice against common opponent responses
- Build confidence before playing online or at a club

3. Product Goals

MVP Goals

1. Teach the London System as White.
2. Teach the Caro-Kann Defense as Black.
3. Provide interactive opening drills.
4. Validate legal moves.
5. Explain the purpose of each move.
6. Track user progress.
7. Offer AI-generated explanations constrained by curated opening content.

Non-Goals for MVP

- Full chess engine training platform
- Online multiplayer
- Tournament prep for advanced players
- Deep Stockfish analysis for every position
- Comprehensive opening database
- Mobile app store release

4. Core User Stories

Learning

As a user, I want to see the basic London System setup so I understand where my pieces belong.

As a user, I want each move explained in plain English so I understand the plan, not just the notation.

As a user, I want to learn common mistakes so I do not lose quickly.

Drilling

As a user, I want the app to play Black responses against my London System so I can practice my moves.

As a user, I want the app to play White responses against my Caro-Kann so I can practice defense.

As a user, I want immediate feedback when I make the wrong move.

Review

As a user, I want to see which opening lines I struggle with.

As a user, I want spaced repetition so weak lines come back more often.

As a user, I want a short post-drill summary showing what I missed.

AI Explanation

As a user, I want to ask, “Why is this move good?” and receive a simple explanation.

As a user, I want the AI to explain strategic themes without hallucinating illegal moves.

As a user, I want the AI to compare the correct move to my mistake.

5. Opening Scope

5.1 London System

Primary line:

1. d4 d5
2. Nf3 Nf6
3. Bf4 e6
4. e3 c5
5. c3 Nc6
6. Nbd2 Be7
7. Bd3 O-O
8. O-O

Key concepts:

- Build a solid pawn center.
- Develop the dark-square bishop early.
- Support the center with e3 and c3.
- Castle safely.
- Prepare Ne5 or e4 depending on Black’s setup.
- Avoid premature attacks before development is complete.

Common Black responses to support:

- ...d5 / ...Nf6 / ...e6 classical setup
- ...c5 early pressure
- ...Bf5 mirror setup
- ...g6 King’s Indian-style setup
- ...c6 Slav-style setup

Common mistakes to teach:

- Moving the same piece repeatedly in the opening
- Attacking before castling
- Ignoring ...Qb6 pressure on b2
- Letting Black trade off the London bishop for free
- Playing h3/g4 attacks too early

5.2 Caro-Kann Defense

Primary line:

1. e4 c6
2. d4 d5

Supported variations:

Advance Variation

1. e4 c6
2. d4 d5
3. e5 Bf5

Key ideas:

- Develop bishop before closing the pawn chain.
- Challenge White's center with ...c5.
- Avoid passive piece placement.

Classical Variation

1. e4 c6
2. d4 d5
3. Nc3 dxe4
4. Nxe4 Bf5

Key ideas:

- Exchange central tension.
- Develop bishop actively.
- Build a solid structure.

Exchange Variation

1. e4 c6
2. d4 d5
3. exd5 cxd5

Key ideas:

- Symmetrical structure.
- Simple development.
- Avoid drifting into passivity.

Fantasy Variation

1. e4 c6
2. d4 d5
3. f3

Key ideas:

- Treat it seriously.
- Strike the center.

- Do not play automatic moves blindly.

Common mistakes to teach:

- Playing ...e6 too early and trapping the light-square bishop
- Ignoring White's central space
- Delaying development
- Accepting a cramped position without counterplay
- Memorizing moves without understanding pawn breaks

6. MVP Feature Set

6.1 Interactive Board

Requirements:

- Render chessboard.
- Support drag-and-drop moves.
- Show legal move highlights.
- Reject illegal moves.
- Display move notation.
- Track current FEN.
- Support board flip for Black drills.

Suggested libraries:

- `react-chessboard`
- `chess.js`

6.2 Lesson Mode

Each lesson contains:

- Opening name
- Position sequence
- Move explanation
- Strategic theme
- Common mistake
- Drill prompt

Example lesson:

Title: London System Basic Setup

Position sequence:

1. d4 d5

2. Nf3 Nf6
3. Bf4

Explanation:

White develops the dark-square bishop before playing e3. This avoids trapping the bishop behind the pawn chain and creates a simple, repeatable setup.

6.3 Drill Mode

The user practices one opening line at a time.

Drill behavior:

1. App presents a position.
2. User makes a move.
3. App validates the move.
4. If correct, app plays opponent response.
5. If incorrect, app gives a hint.
6. After repeated failure, app reveals the correct move.
7. Drill result is saved.

Feedback levels:

- First miss: general hint
- Second miss: strategic hint
- Third miss: show correct move and explanation

6.4 AI Coach Panel

The AI coach answers constrained questions:

Supported questions:

- Why is this move good?
- What is the plan here?
- What did I do wrong?
- What should I watch out for?
- What is Black/White threatening?

AI guardrails:

- AI must not determine move legality.
- AI must not invent unsupported lines.
- AI must cite the current board state and known lesson content.
- AI must defer to Stockfish/chess.js for rules and tactical evaluation.
- AI should speak plainly and avoid grandmaster jargon.

6.5 Progress Tracking

Track:

- Lessons completed
- Drill attempts
- Correct first attempts
- Repeated mistakes
- Opening line confidence score
- Last practiced date

Simple scoring:

- Correct first try: +3
- Correct after hint: +1
- Incorrect after reveal: -1
- Not practiced recently: decay score

7. Technical Architecture

7.1 Stack

Frontend:

- Next.js
- TypeScript
- React
- Tailwind CSS
- shadcn/ui
- react-chessboard
- chess.js

Backend:

- Next.js App Router API routes
- Server Actions where appropriate
- PostgreSQL
- Prisma ORM

AI:

- Local Gemma 4 through local inference endpoint
- Optional OpenAI-compatible local API wrapper
- Retrieval from curated opening content
- No direct AI authority over legal move validation

Chess engine:

- chess.js for legal moves
- Optional Stockfish WASM for evaluation
- Engine use should be explainability support, not required for MVP

7.2 Suggested Architecture

Modules:

- `opening-content`
- `chess-rules`
- `drill-engine`
- `ai-coach`
- `progress-tracking`
- `lesson-ui`
- `admin-content-editor`

Core principle:

The deterministic chess system owns correctness.
The AI owns explanation.

8. Data Model

8.1 Opening

Fields:

- id
- name
- side: white | black
- description
- difficulty
- createdAt
- updatedAt

8.2 Line

Fields:

- id
- openingId
- name
- startingFen
- moveSequencePgn

- description
- priority
- active

8.3 MoveExplanation

Fields:

- id
- lineId
- moveNumber
- san
- fenBefore
- fenAfter
- explanation
- theme
- commonMistake
- hintLevel1
- hintLevel2
- revealExplanation

8.4 DrillAttempt

Fields:

- id
- userId
- lineId
- moveExplanationId
- attemptedMove
- expectedMove
- correct
- hintUsed
- createdAt

8.5 UserProgress

Fields:

- id
- userId
- openingId
- lineId
- confidenceScore
- completedLessons
- lastPracticedAt

9. AI Prompting Strategy

9.1 System Prompt

You are a chess opening coach for adult returning players. Explain moves clearly and practically. Do not invent opening lines. Do not claim a move is legal unless it is provided as legal by the chess engine. Use the supplied board state, legal moves, opening lesson, and evaluation data. Prefer plain English over chess jargon.

9.2 AI Context Payload

The AI receives:

- Opening name
- Current FEN
- Legal moves from chess.js
- Expected move
- User attempted move
- Lesson explanation
- Common mistake
- Optional Stockfish evaluation
- User skill level

The AI does not receive authority to mutate board state.

9.3 Example AI Request

User asks:

Why is Bf4 played before e3 in the London?

AI response should explain:

In the London System, White usually develops the dark-square bishop to f4 before playing e3 because e3 would otherwise block the bishop inside the pawn chain. Bf4 gets the bishop active early, supports the center, and creates a repeatable setup.

10. Demo Flow

10.1 Demo Narrative

“This is not a chess app with AI sprinkled on top. It is a deterministic training system where AI is constrained to explanation. That makes it a better example of practical AI-assisted software design.”

10.2 Live Demo Steps

1. Show landing page.
2. Select London System.
3. Start basic setup lesson.
4. Make correct moves.
5. Intentionally make a bad move.
6. Show hint system.
7. Ask AI coach why the move was wrong.
8. Switch to Caro-Kann.
9. Practice against the Advance Variation.
10. Show progress dashboard.
11. Show code architecture.
12. Show where AI is allowed and where it is not.

10.3 Demo Talking Points

- Domain logic is deterministic.
- AI is useful but constrained.
- The app separates correctness from explanation.
- The chess engine prevents hallucinated moves.
- The curated content gives the model reliable context.
- This pattern applies directly to enterprise AI systems.

11. UI Pages

11.1 Home

Sections:

- Choose your repertoire
- London System card
- Caro-Kann card
- Continue practice
- Progress summary

11.2 Opening Overview

Sections:

- Opening summary
- Key ideas
- Main setup
- Common mistakes
- Start lesson
- Start drills

11.3 Lesson Page

Layout:

- Chessboard left
- Explanation panel right
- Move list below
- AI coach drawer
- Next / Previous controls

11.4 Drill Page

Layout:

- Chessboard
- Prompt
- Feedback panel
- Hint button
- Progress indicator
- “Explain this” AI button

11.5 Progress Page

Sections:

- Overall confidence
- London System progress
- Caro-Kann progress
- Weak lines
- Suggested next drill

12. Acceptance Criteria

12.1 Chess Board

- User can make legal moves.
- Illegal moves are rejected.
- Board state updates correctly.
- Move notation is displayed.
- Board can flip for Black.

12.2 London System Lesson

- User can complete the basic London setup.
- Each move has an explanation.
- At least three common Black responses are supported.

12.3 Caro-Kann Lesson

- User can practice 1.e4 c6 2.d4 d5.
- Advance, Classical, and Exchange variations are supported.
- Each variation includes explanation and common mistakes.

12.4 Drill Engine

- Correct moves advance the drill.
- Incorrect moves produce hints.
- Repeated incorrect moves reveal the correct answer.
- Drill attempts are persisted.

12.5 AI Coach

- AI receives current board state.
- AI uses curated lesson content.
- AI does not generate illegal moves as recommendations.
- AI explains mistakes in plain English.
- AI response is logged for debugging.

13. Future Enhancements

- Stockfish evaluation overlay
- PGN import
- Custom repertoire builder
- Spaced repetition calendar
- Puzzle mode from opening positions
- Voice narration
- YouTube demo export mode
- Local-only privacy mode
- Multiplayer sparring bot
- Coach personality settings
- “Explain like I’m 1200 rated” mode
- “Enterprise Architect mode,” where every chess mistake is blamed on governance

14. Risks

14.1 AI Hallucination

Risk:

The model may invent chess lines or give unsound advice.

Mitigation:

Use chess.js and Stockfish as sources of truth. Limit AI to explanation.

14.2 Opening Content Accuracy

Risk:

Curated opening lines may contain mistakes.

Mitigation:

Represent every line as PGN and validate through chess.js during seed import.

14.3 Scope Creep

Risk:

Chess apps can become endless.

Mitigation:

Limit MVP to London System and Caro-Kann.

14.4 Overbuilding Engine Features

Risk:

Trying to compete with Lichess or Chess.com.

Mitigation:

Focus on guided learning and AI explanation, not full chess platform features.

15. Recommended MVP Build Order

1. Create Next.js app.
2. Add chessboard component.
3. Add chess.js move validation.
4. Seed London and Caro-Kann lines.
5. Build lesson viewer.
6. Build drill engine.
7. Add progress persistence.
8. Add AI coach endpoint.
9. Add constrained prompt template.
10. Add dashboard.
11. Polish demo script.

12. Record demo.

16. Definition of Done

The MVP is complete when a user can:

- Learn the London System basic setup.
- Practice the London System against common Black responses.
- Learn the Caro-Kann basic structure.
- Practice Caro-Kann responses to common White variations.
- Receive hints after mistakes.
- Ask the AI coach for explanations.
- See progress over time.
- Trust that legal move validation is deterministic, not guessed by AI.